

Unit and Integration Testing

- Testing
- Unit Testing
- Integration Testing

Why Testing is Important

Testing is an important part of the development cycle. Its main purpose is to verify that the software's desired behaviour is correct and to identify any defects.

Such defects might include:

- Errors
- Warnings
- Algorithm mistakes
- Functionality mistakes

Why Testing is Important

- Testing's main goal is to eliminate defects in software before they become much larger problems or problems that end-users see.
- If issues or bugs in smaller components are missed they become much harder to track down and fix.
- When issues get identified later during development the fix often only gets applied to the current projects code base. This means that the fix isn't applied to the original library or source. When the original code gets reused it will most likely still have the same issues which will have to be re-fixed in the next project.

Test Driven Development

In most outlines of life cycles the testing phase is in between the development and release phase. In reality testing should be considered throughout the entirety of the software lifecycle.

Software based testing is typically designed during the design and implementation stages of the software life cycle. Test are designed and implemented every time a new section of code needs to be written.

The test phase of development should be used to test code and fix any remaining issues or bugs and not to retrofit unit and integration tests to the existing code base.

Type of Testing

- Unit tests
 - Looks at testing small individual parts such as functions, components and classes
 - Can be automated
- Integration tests
 - Looks at how components of a program behave together to eliminate/spot any irregularities
 - Can be automated
- UI tests
 - Normally involve a series of test scenarios and outcomes that focus on if an action gets performed correctly rather than how the underlying code performed it.
 - Are hard or impossible to automate and are done manually
- Requirement and Functional Requirement testing
 - Looks at, does what we are building solve our client's problem
 - Does the created software match the functional requirements

After functions, components and classes are integrated into larger systems, errors and bugs become much harder to track down and fix.

To avoid this problem smaller parts of a self-contained code should be unit tested.

Unit testing is the process of testing individual functions, components and classes to remove as many errors or bugs before they are integrated into a larger system.

Unit Testing may

- Allows programmers to validate if the smallest parts of their code base work and behave as expected.
- Insure that any changes made to the smallest parts of the program don't result in a change to the expected behaviour.
- Insures that if any changes are made to the smallest parts of the program that no new errors or issues are introduced

Planning Unit Testing

- 1) Identify the requirements for a new function class or component
 - a. Expected behaviour
 - b. Input types
 - c. Expected output
 - d. Any required try/catch conditions
 - e. Expected fail cases where the function is not suitable to use
- 2) Design test cases
- 3) Write test code
- 4) Write the new function class or component
- 5) Perform tests to ensure the new function behaves correctly

Unit Testing and Test Cases

Testing a function, component or class with only one set of inputs and one expected output is not capable of eliminating unforeseen bugs or issues.

Test cases are a set of tests for a single function, component or class that puts it through a series of different scenarios to ensure that a function, component or class works correctly under in different conditions. These scenarios are known as a test case.

A single test case contains a set of:

- The function component or class being tested
- A description of the test case
- The input used in the test case
- The expected outcome of the component or class being tested

Unit Testing and Test Cases

Example of test cases of a function that adds two number together:

Description	Input	Outcome
Missing Input	Parameter one is 22 parameter two is missing but it has a default of 2	The function preforms correctly and returns 24
Bad input	Parameter one is 22 parameter two is "2"	The function preforms correctly and returns 24 "2" is cast to the int 2
File system not found	Parameter one is 22 parameter two is "2"	The file system could not be written and an error is thrown
Triggers validation	Parameter one is 22 parameter two is "2"	The function throws an error. "parameter 2 must be an int"

- Methods, functions and classes may work correctly on their own but however may have unforeseen issues or bugs when used in conjunction with other parts of the code base.
- Integration tests are designed to test how well several methods, functions and classes behave together. Integration tests are used to make sure that changes to components or modules don't adversely affect the rest of the program.
- The most common use of integration testing in web development is testing of controller functions and API calls.

Integration testing has three main benefits:

- 1) As they are based off functional requirements they can be used to verify if the functional requirement match the final produced product
- 2) They also help insure that changes made to the code base don't introduce new bugs or errors.
- 3) They are also capable of finding issue that sub components have when they interact with each other. As unit test can only test functions, components and classes in isolation they aren't able to identify these sort of issues.

Integration tests are normally much larger than unit tests and can have a number of extra stages.

A single test cases with in an integration test may have to set up the database with specific entries in order for the integration test to function and produce correct results.

Integration Testing and Test Cases

Example of Integration testing cases:

Description	Data requirements	Input Data	Expected output	Data clean up requirements
Missing input on create client API call triggers input validation	No data required	Name="" Age="22" State="QLD"	Json returns with "the name field is required"	No clean up required
All input available & db insert of user	No existing users should be present in the user table	Name="Joe" Age="22" State="QLD"	JSON data returns true and the new user in the database exists	Remove added user from users table in the database

Planning Integration Tests

- 1) Define all functional requirements
- 2) For each functional requirement a new set of test cases are created with a new function
- 3) For each test create several test cases - there should be a test for:
 - a. All data correct
 - b. Validation reject
 - c. Correct data returned
 - d. All try/catch blocks are triggered and are handled correctly
- 4) For each test case identify any test data requirements and any clean-up activities required for the next test to work correctly
- 5) Write test code
- 6) Write code
- 7) Run tests to insure all written code works as expected

The MEAN stack and Automated Test Tools

- Running tests manual is a very long, boring and expensive undertaking. Automated testing frameworks provide a way to quickly test code and reduce the possibility of human error. Both Unit and Integration testing can be automated in this way.
- Node JS has many testing frameworks. Some of the more popular ones include:
 - Expresso
 - Mocha
 - Intern
 - Vows
 - Node unit
 - Js Unit
 - Node Tap
- The MEAN stack uses Mocha as its unit testing framework.